

Artificial Intelligence and Machine Learning(6CS012)

Part - II: Vision Tasks

Name: Mandeep Panta

Student Id: 2408214

Lecturer/Tutor: Ms. Sunita Parajuli

Module Leader: Mr. Siman Giri

Submission Date: 2026-05-10

Table of Contents

Introduction	1
Dataset.....	2
Dataset Characteristics:.....	2
Methodology.....	4
Baseline CNN Model	4
Deeper CNN Model with Regularization	5
Training with Different Optimizers (Deeper Model)	6
Transfer Learning (MobileNetV2).....	6
Experiments and Results	7
Baseline vs. Deeper Architecture.....	7
Computational Efficiency	7
Training with Different Optimizers	9
Challenges in Training.....	10
Fine-Tuning or Transfer Learning	11
Performance Improvements and Comparison:	11
Conclusion and Future Work.....	13

Abstract

This project focuses on multi-class traffic sign classification, an essential task in autonomous driving and intelligent transportation systems. The objective was to

develop and compare different Convolutional Neural Network (CNN) architectures to classify traffic signs into five categories: Cautions, Crossings, Direction, No Entry, and Speed Limit.

A baseline CNN model was first implemented, followed by a deeper CNN architecture enhanced with Batch Normalization, Dropout, and L2 regularization to improve learning and reduce overfitting. Different optimizers, including SGD and Adam, were evaluated, and an ablation study highlighted the importance of Dropout in improving model generalization. Transfer learning using MobileNetV2 was also explored through feature extraction and fine-tuning.

The deeper CNN achieved the best performance with a validation accuracy of 99.88%, outperforming the baseline model's 99.72%. Results showed that regularization techniques significantly improved stability and reduced overfitting. Although transfer learning is often effective, MobileNetV2 achieved only 50.98% accuracy, likely due to differences between ImageNet features and the simpler traffic sign dataset, along with the smaller image resolution.

Overall, the study demonstrates that carefully designed CNN architectures with proper regularization can achieve highly accurate traffic sign recognition, emphasizing the importance of model design, optimizer selection, and dataset-specific tuning in deep learning applications.

Introduction

Traffic Sign Recognition is a fundamental component of advanced driver-assistance systems and autonomous driving technologies. Accurate and real-time classification of traffic signs is paramount for ensuring road safety, efficient traffic flow, and effective navigation. Errors in recognition can lead to hazardous situations, making the development of robust and highly accurate classification models a critical area of research.

Deep learning, particularly Convolutional Neural Networks (CNNs), has revolutionized image classification tasks due to its ability to automatically learn hierarchical features from raw image data. CNNs have demonstrated state-of-the-art performance across a wide range of visual recognition challenges, making them an ideal choice for this project.

This project aims to address the multi-class classification of Traffic Signs. We will explore and compare different CNN architectures, investigate the impact of regularization techniques, evaluate various optimizers, and assess the benefits of transfer learning. The goal is to identify an optimal model configuration that achieves high accuracy and robust generalization for traffic sign recognition.

Dataset

The dataset used for this project is a multi-class traffic sign image classification dataset, provided as a zip file named Traffic_Sign_-2.zip. It is an everyday road sign dataset, with each image belonging to one of five semantic categories:

- Cautions
- Crossings
- Direction (folder name DIRECTION)
- No Entry
- Speed Limit

Dataset Characteristics:

- Source: Dataset was provided by Tutor Ms. Sunita Parajuli.
- Number of Images: The training directory initially contained 16,065 images across all classes.
- Classes: 5 distinct classes as listed above.
- Resolution: All images are resized to 96 × 96 pixels (IMG_HEIGHT = 96, IMG_WIDTH = 96).
- **Preprocessing:**
 - Corrupted Image Removal: A custom function `remove_corrupted_images` was applied to identify and remove any corrupted image files from the dataset. In total, 35 corrupted images were removed from the training directory.
 - Resizing: Images are resized to (96, 96) pixels.
 - Normalization: Pixel values are normalized to the range [0, 1] by dividing by 255.
- Data Augmentation: A `data_augmentation` layer was applied, including `RandomRotation (0.05)`, `RandomZoom (0.1)`,

and `RandomContrast (0.1)`. Notably, horizontal flipping was explicitly avoided to preserve the semantic meaning of directional signs.

- Train/Validation Split: The dataset is split into training, and validation sets with a 0.2 (20%) validation split. This split is stratified to maintain similar class proportions in both sets, reflecting the natural class imbalance.

Class Distribution

- Cautions: 1671 images
- Crossings: 1821 images
- Direction: 2961 images
- No Entry: 2931 images
- SpeedLimit: 6681 images

This distribution highlights a class imbalance, with 'SpeedLimit' being the most frequent category. To address this, class weights were computed and applied during model training to ensure fair learning across all classes.

Methodology

This section details the deep learning models, architectural designs, and training configurations employed in the project.

Baseline CNN Model

- Architecture: A simple sequential CNN model designed from scratch.
 - Input Layer: `Input(shape=(96, 96, 3))`
 - Data Augmentation: Applied as the first layer.
 - Convolutional Blocks (3): Each block consists of a `Conv2D` layer with `relu` activation and `padding='same'`, followed by `MaxPooling2D((2,2))`. The filter counts are 32, 64, and 128 for Block 1, 2, and 3 respectively.
 - Flattening: `GlobalAveragePooling2D()` is used to flatten the feature maps.
 - Fully Connected Layers (3): Dense layers with `relu` activation and units 256, 128, and 64.
 - Output Layer: `Dense(NUM_CLASSES, activation='softmax')` for 5-way classification.
- Loss Function: `sparse_categorical_crossentropy`
- Optimizer: Adam with `learning_rate=1e-3`.
- Hyperparameters:
 - `IMG_HEIGHT = 96`, `IMG_WIDTH = 96`
 - `BATCH_SIZE = 32`
 - `SEED = 42`
 - `EPOCHS = 20` (with `EarlyStopping`)

Deeper CNN Model with Regularization

- Architecture: An extended version of the baseline model, featuring increased depth, wider layers, and various regularization techniques.
 - Input Layer: `Input(shape=(96, 96, 3))`
 - Data Augmentation: Applied as the first layer.
 - Convolutional Blocks (4): Each block contains two Conv2D layers with `padding='same'`, BatchNormalization, and relu activation, followed by MaxPooling2D((2,2)) and Dropout. Filter counts increase through blocks: 32 (Block 1), 64 (Block 2), 128 (Block 3), 256 (Block 4).
 - Regularization:
 - BatchNormalization after each Conv2D layer.
 - Dropout layers with rates 0.25, 0.30, 0.35, 0.40 after each MaxPooling2D layer.
 - L2 weight penalty ($1e-4$) applied to `kernel_regularizer` in all Conv2D and Dense layers.
 - Flattening: `GlobalAveragePooling2D()`.
 - Fully Connected Layers: Dense layers with relu activation and units 256 and 128, each followed by Dropout (0.5).
 - Output Layer: `Dense(NUM_CLASSES, activation='softmax')`.
- Loss Function: `sparse_categorical_crossentropy`.
- Optimizer: Adam with `learning_rate=1e-3` (for primary training).
- Hyperparameters: Same as baseline, with additional callbacks:
 - EarlyStopping: `monitor='val_loss'`, `patience=3`, `restore_best_weights=True`.
 - ReduceLROnPlateau: `monitor='val_loss'`, `factor=0.5`, `patience=3`.

Training with Different Optimizers (Deeper Model)

The deeper model was also trained with two different optimizers for comparison:

- SGD Optimizer: `keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)`.
- Adam Optimizer: `keras.optimizers.Adam(learning_rate=1e-3)` (for comparison with SGD in a separate run)

Transfer Learning (MobileNetV2)

- Pre-trained Model: MobileNetV2, pre-trained on ImageNet, with `include_top=False` to remove its classification head.
- Architecture Modification:
 - The `base_model` (MobileNetV2) was added.
 - `GlobalAveragePooling2D()` was added.
 - Two Dense layers (256, 128 units) with `relu` activation and Dropout (0.5, 0.3) were added.
 - An Output Layer: `Dense(NUM_CLASSES, activation='softmax')`.
- Training Strategy:
 - Feature Extraction: Initially, the `base_model` layers were frozen (`base_model.trainable = False`), and only the newly added classification head was trained.
 - Fine-Tuning: Subsequently, the `base_model` was unfrozen (`base_model.trainable = True`), and the lower 30 layers of MobileNetV2 were frozen. The entire model was then recompiled with a lower learning rate (1e-5) for fine-tuning.
- Loss Function: `sparse_categorical_crossentropy`.
- Optimizer: Adam (`learning_rate=1e-3` for feature extraction, 1e-5 for fine-tuning).
- Hyperparameters: Same callbacks as the Deeper CNN (`EarlyStopping`, `ReduceLROnPlateau`).

Experiments and Results

Baseline vs. Deeper Architecture

Accuracy and Loss Comparison:

Metric	Baseline	Deeper Model
Validation Accuracy	0.9972	0.9988
Validation Loss	0.0029	0.0029

- **Accuracy Improvement:** The Deeper CNN model achieved a marginal but consistent accuracy improvement of **0.0016** over the Baseline CNN. This indicates that the added architectural complexity and regularization helped the model learn more robust features and generalize slightly better on the validation set.
- **Loss Curves:** Both models exhibited very low validation loss, suggesting effective learning. The Deeper model, with its regularization, generally showed a slightly more stable validation loss curve, less prone to the rapid fluctuations or early rises often indicative of overfitting in the Baseline model.

Computational Efficiency

Comparison of Training and Inference Costs:

Metric	Baseline (Model)	Deeper Model	Difference
Training Time (sec)	113.41	51.54	-61.87
Inference Time (sec)	1.28	1.24	-0.04
Memory Usage (MB)	18.51	18.51	0.00

- **Training Time:** Surprisingly, the Deeper Model's training time (51.54 seconds) was significantly *lower* than the Baseline Model's (113.41 seconds) for the

reported run. This is counter-intuitive given its increased complexity and parameter count, but can be attributed to the EarlyStopping callback. The Deeper Model likely converged faster or reached its optimal val_loss earlier than the Baseline, thus stopping training epochs sooner.

- **Inference Time:** The inference times were very similar, with the Deeper Model being marginally faster. Both models are efficient for prediction.
- **Memory Usage:** The reported memory usage (proxy for parameter count) for both models was identical, which is incorrect as the deeper model has more parameters. This indicates an error in how `model.count_params()` was applied, possibly counting the parameters of the *same* model instance or overwriting the model variable.
- **Hardware Acceleration:** All experiments were conducted using GPU acceleration available in Google Colab, which significantly sped up the training process for both architectures.

Training with Different Optimizers

For the Deeper CNN model, two different optimizers were compared: SGD and Adam.

Optimizer Comparison:

Metric	SGD	Adam
Validation Accuracy	0.6044	0.2593
Training Time (sec)	93.11	100.11

- **Convergence Speed:** The Adam optimizer, in this specific comparison run, took slightly longer (100.11 sec) than SGD (93.11 sec) to train.
- **Final Accuracy:** SGD significantly outperformed Adam in this particular comparison, achieving a validation accuracy of **0.6044** compared to Adam's **0.2593**. This result is contrary to typical expectations where Adam often converges faster and achieves higher accuracies, especially in deeper networks. The discrepancy from the narrative (Validation Accuracy | 59.45% | 74.17%) indicates that the model for the Adam optimizer might have encountered an issue during this particular training run, leading to poor performance, or the callbacks were interfering in a way that resulted in poor final model performance. However, based on the *executed outputs*, SGD performed better in this comparison.
- **Class-wise Performance (SGD):** SGD showed reasonable performance for 'Direction' (0.8459 F1-score) and 'SpeedLimit' (0.6667 F1-score), but struggled with 'Cautions' (0.3300 F1-score) and 'Crossings' (0.1341 F1-score).

Challenges in Training

Several challenges were encountered during the training process, leading to important observations:

- **Overfitting:** The initial deeper model, particularly without proper regularization (as shown in the ablation study), quickly overfit the training data. Training accuracy would reach near 100%, while validation accuracy stagnated or declined. This was effectively mitigated by incorporating Dropout, Batch Normalization, L2 Regularization, and EarlyStopping.
- **Underfitting:** The training with the SGD optimizer, in its compared run, exhibited signs of underfitting, with a relatively low validation accuracy and poor performance on certain classes. This suggests that the model failed to learn sufficiently complex patterns with the given SGD configuration in that specific run.
- **Class Imbalance:** The inherent class imbalance in the dataset (e.g., 'SpeedLimit' having many more samples than 'Cautions') was addressed by computing and applying `class_weights` during model training. This helped the models give more importance to minority classes, preventing them from being overshadowed by majority classes.
- **Optimizer Sensitivity:** The significant difference in performance between SGD and Adam in the specific comparison run (where Adam underperformed) highlights the sensitivity of model training to optimizer choice and its hyperparameters. While Adam is generally robust, specific initializations or interactions with callbacks can lead to suboptimal outcomes.
- **Total Training Time:** The total wall-clock time for training all models (Deeper, SGD, Adam, Ablation) combined was **240.99 seconds (approximately 4.02 minutes)**, utilizing GPU acceleration.

Fine-Tuning or Transfer Learning

To explore the benefits of leveraging pre-trained knowledge, **MobileNetV2** was chosen as the base model for transfer learning.

Choice of Pre-trained Model: MobileNetV2 was selected due to its lightweight nature, computational efficiency, and strong performance on image classification tasks. It is pre-trained on the large-scale ImageNet dataset and is well-suited for deployment on resource-constrained environments. For traffic signs, which often feature simple geometric patterns and color information, MobileNetV2's efficient feature extraction capabilities were considered advantageous.

Approach:

1. **Feature Extraction:** The convolutional base of MobileNetV2 was loaded with imagenet weights and its top classification layers removed (`include_top=False`). The `base_model` was initially frozen (`trainable=False`), and a new classification head (consisting of `GlobalAveragePooling2D`, two `Dense` layers with `relu` activation and `Dropout`, and a `softmax` output layer) was added and trained.
2. **Fine-Tuning:** After initial training of the head, the `base_model` was unfrozen (`trainable=True`). To prevent disruption of well-learned low-level features, the initial layers of MobileNetV2 were refrozen (`base_model.layers[:30]`). The model was then recompiled with a very low learning rate ($1e-5$) and further trained (fine-tuned).

Performance Improvements and Comparison:

Model	Accuracy
Baseline CNN	0.9972
Deeper CNN	0.9988
Transfer Learning	0.5098

- **Validation Accuracy:** The transfer learning model achieved a validation accuracy of **0.5098**. This is a significantly lower performance compared to both the Baseline CNN (0.9972) and the Deeper CNN (0.9988) trained from scratch.
- **Discussion:** The unexpectedly low performance of the MobileNetV2 transfer learning model suggests that simply using a pre-trained model is not a guaranteed improvement. Potential reasons for this outcome include:
 - **Dataset Disparity:** The nature of traffic sign images (often clear, simple shapes, specific colors) might be very different from the complex, varied images in ImageNet, making the pre-learned features less directly transferable.
 - **Image Resolution:** MobileNetV2 is typically designed for higher resolution inputs (e.g., 224x224). Applying it directly to 96x96 images might lead to a loss of detail critical for its learned features, requiring more aggressive fine-tuning or a different pre-processing approach (e.g., upsampling inputs).
 - **Hyperparameter Tuning:** The fine-tuning process, especially the unfreezing strategy and learning rate, might not have been optimal for this specific task.
 - **Callack Interference:** Similar to the Adam optimizer comparison, interactions with EarlyStopping or ReduceLROnPlateau could have influenced the final recorded accuracy if the model stopped training prematurely due to a local minimum or plateau in validation loss, before fully adapting to the dataset.

Conclusion and Future Work

This project successfully developed and evaluated several Convolutional Neural Network (CNN) architectures for multi-class traffic sign classification. Key findings include:

- **Deeper Architectures Outperform Baseline:** The Deeper CNN model, incorporating Batch Normalization, Dropout, and L2 regularization, achieved a validation accuracy of **0.9988**, a marginal but consistent improvement over the Baseline CNN's **0.9972**. This demonstrates the effectiveness of increased architectural depth and regularization in learning robust features and improving generalization.
- **Importance of Regularization:** The ablation study unequivocally highlighted the critical role of Dropout. Its removal led to a significant performance drop of approximately **31.15%** (from 0.9988 to 0.6872 in the specific ablation run), underscoring its necessity in mitigating overfitting and ensuring stable generalization in deeper networks.
- **Optimizer Impact:** While Adam is generally preferred for its adaptive learning rates and faster convergence, the specific comparison run showed SGD outperforming Adam (0.6044 vs 0.2593 validation accuracy). This unusual result suggests a potential issue in that specific Adam training run (e.g., interaction with callbacks, suboptimal learning rate schedule for that particular run), contrasting with the general understanding of Adam's efficacy. However, the initial Deeper model, which used Adam, achieved very high accuracy.
- **Challenges with Transfer Learning:** The MobileNetV2-based transfer learning model yielded a surprisingly low validation accuracy of **0.5098**, significantly underperforming models trained from scratch. This indicates that pre-trained models are not universally superior and require careful adaptation, hyperparameter tuning, and consideration of dataset characteristics (e.g., image resolution, domain similarity to ImageNet).

Trade-offs Between Different Architectures and Methods

- **Complexity vs. Performance:** Deeper models offer higher accuracy and better feature learning but come with increased computational costs (training time, memory usage). The early stopping callback mitigated the increased training time for the Deeper model by stopping when validation loss plateaued or increased.
- **Regularization vs. Overfitting:** Techniques like Dropout, Batch Normalization, and L2 regularization are crucial for deeper models to prevent overfitting, but their absence can severely degrade performance.
- **Optimizer Choice:** The choice of optimizer greatly influences convergence speed and final model performance. While Adam is often robust, SGD can perform well with proper tuning and in some scenarios.
- **Transfer Learning vs. Training from Scratch:** Transfer learning can provide a powerful head start, but its effectiveness depends heavily on the similarity between the pre-training and target datasets, as well as meticulous fine-tuning. For unique datasets or specific image characteristics, training a custom CNN from scratch might be more effective.

Limitations and Failure Cases

- **Dataset Specificity for Transfer Learning:** The poor performance of MobileNetV2 suggests that the pre-trained features from ImageNet might not be optimally suited for the distinct visual characteristics of traffic signs at a 96x96 resolution. The low resolution might lose details critical for MobileNetV2's pre-learned features.
- **Optimizer Anomaly:** The observed underperformance of Adam compared to SGD in one specific comparison run for the deeper model is an anomaly that needs further investigation. It might be due to a specific interaction with early stopping or learning rate scheduling in that particular execution.
- **Class Imbalance Impact:** While class weights were applied, minority classes still showed slightly lower recall and F1-scores in some evaluations, indicating that balancing strategies need to be continually refined.

Potential Improvements and Future Research Directions

- **Advanced Data Augmentation:** Explore more sophisticated augmentation techniques such as Mixup, CutMix, or AutoAugment to further increase dataset diversity and model robustness.
- **Hyperparameter Optimization:** Conduct a more rigorous hyperparameter search (e.g., using techniques like Grid Search, Random Search, or Bayesian Optimization) for learning rates, dropout rates, and L2 regularization strengths across all models.
- **Ensemble Methods:** Combine predictions from multiple models (e.g., Baseline, Deeper, and even fine-tuned transfer learning models if optimized) to potentially achieve higher accuracy and robustness.
- **Alternative Transfer Learning Models:** Experiment with other lightweight pre-trained models (e.g., EfficientNet variants, ResNet) or pre-training strategies optimized for smaller image sizes.
- **Higher Resolution Inputs:** Investigate the impact of training models with higher-resolution images (e.g., 224x224) if computational resources allow, especially for transfer learning models that are typically designed for such inputs.
- **Real-time Performance Optimization:** Further optimize models for deployment in real-time embedded systems by exploring quantization, pruning, or knowledge distillation techniques.

By addressing these areas, the traffic sign classification system can be made even more robust, accurate, and efficient for real-world advanced driver-assistance systems and autonomous driving applications.